The next listing shows the final version of the tests with dependencies avoided where possible and injected where necessary.

Listing 10.2. Testing with a double

```
assert = require 'assert'

fact = (description, fn) ->
  try
    fn()
    console.log "#{description}: OK"
  catch e
    console.log "#{description}: \n#{e.stack}"

http =
  get: (options, callback) ->
    callback "canned response"
    @
  on: (event, callback) ->

fetch = (topic, http, callback) ->
  options =
    host: 'www.agtronscameras.com'
    port: 80
    path: "/data/#{topic}"

  http.get(options, (result) ->
    callback result
  ).on 'error', (e) ->
    console.log e


parse = (data) ->
  "parsed canned response"

fact "data is parsed correctly", ->
  parsed = parse 'canned response'
  assert.equal parsed, "parsed canned response"

fact "data is fetched correctly", ->
  fetch "a-topic", http,  (result) ->
    assert.equal result, "canned response"
```

The http double to be used by tests

Very simple parse function shown to demonstrate it tested in isolation from fetch

When you isolate dependencies and inject them only into places where you absolutely need them, the program is easier to reason about and easier to test. Dependency injection can lead to a particular problem, though. In a